

Enunciado do Projeto Final

Agente Explorador na Arena 3D RAG

Índice

1. Introdução e Contexto da Missão
 2. Arquitetura Global do Sistema
 3. Preparação do Ambiente de Desenvolvimento (Setup)
 4. A Física da Arena: Regras e Mecânicas
 5. O Ciclo de Vida do Agente (Percepção, Decisão e Ação)
 6. O Desafio RAG: Terminais de Plasma e LLM
 7. Regras de Ouro do Servidor e Sobrevivência
 8. Modos de Operação
 9. Fases de Entrega e Critérios de Avaliação
-

1. Introdução e Contexto da Missão

1.1. O Cenário: A Arena Cibernética de 2026

Bem-vindos ao projeto final da unidade curricular. Neste desafio, vão entrar na Arena 3D RAG, um ambiente de simulação e avaliação hospedado num servidor de produção Cloud (**arena.pmonteiro.ovh**). Esta arena é um laboratório hostil e competitivo onde múltiplos agentes autónomos são largados para explorar labirintos, recolher artefactos de energia e contornar obstáculos táticos em tempo real.

No entanto, este não é um simples jogo de labirintos. Espalhados pela arena encontram-se Terminais de Plasma (Cofres) altamente encriptados. Para abrir estes terminais e concluir as missões, não basta usar a força bruta ou a navegação mecânica; o vosso robô terá de ler, interpretar e raciocinar sobre manuais técnicos de engenharia em tempo real para encontrar as chaves de descriptação.

1.2. O Vosso Papel como Engenheiros de IA

Vocês não vão controlar o robô com um teclado ou comando. O vosso papel é atuar como Engenheiros de Sistemas Cognitivos. Terão a responsabilidade de programar o “cérebro” do robô (um cliente construído em Java) que irá funcionar de forma 100% autónoma.

O vosso software deverá conectar-se via REST API ao servidor central da Arena, receber dados de telemetria a cada turno (o radar do agente, o seu nível de bateria, a localização de paredes e inimigos) e decidir de forma algorítmica qual a próxima ação motora a tomar (MOVER_NORTE, MOVER_SUL, etc.), sobrevivendo ao desgaste energético e a possíveis ataques de robôs rivais.

1.3. O Paradigma do Agente: Heurística e IA Generativa

O verdadeiro desafio deste projeto é a construção de uma Arquitetura Híbrida. O cérebro do vosso agente será dividido em dois hemisférios:

O Motor Determinístico (Heurística): Responsável por processar a matriz espacial, construir um mapa de calor na memória do agente, evitar bater em paredes, fugir de inimigos se a bateria estiver baixa e focar a trajetória nos recursos mais próximos.

O Motor Semântico (RAG - Retrieval-Augmented Generation): O ex-líbris do vosso robô. Quando o agente pisa um Terminal de Plasma, o servidor apresenta-lhe um enigma em linguagem natural. O vosso programa Java terá de interrogar um modelo de Inteligência Artificial local (usando o motor Ollama e o modelo Small Language Model **qwen2.5-coder:0.5b-instruct**) para varrer um manual técnico (

GET

/arena/{room_id}/download_manual

Download Active Manual)

, extrair a secção correta através de cálculos vetoriais (Cosine Similarity) e aplicar Prompt Engineering para obrigar a IA a cuspir o código de desbloqueio exato, sem alucinações.

1.4. Objetivos Pedagógicos

Ao concluírem este projeto com sucesso, terão consolidado competências práticas exigidas na vanguarda da indústria de Software:

- **Integração de Sistemas Distribuídos:** Comunicação cliente-servidor robusta através de chamadas HTTP/REST e processamento contínuo de ficheiros JSON.
- **Resiliência de Rede:** Lidar com timeouts, bloqueios de servidor (Anti-Flood / DDOS protection) e sincronização de threads (Thread.sleep).
- **Bases de Dados Vetoriais (Embeddings):** Transformar texto humano em vetores matemáticos para pesquisas semânticas de alta precisão.
- **Engenharia de Prompts (ChatML):** Dominar um modelo fundacional (LLM), utilizando restrições negativas e delimitação de contexto para forçar outputs determinísticos (JSON/Chaves) a partir de IA generativa.

Preparem as vossas IDEs, iniciem os vossos motores locais do Ollama e que vença o algoritmo mais inteligente!

2. Arquitetura Global do Sistema

Para que este projeto seja exequível e realista, o ecossistema da Arena 3D RAG foi desenhado segundo uma **Topologia Híbrida (Cloud + On-Premise)**. Isto significa que o processamento do jogo e o raciocínio da Inteligência Artificial estão fisicamente separados.

O sistema assenta em três pilares fundamentais:

2.1. O Servidor Central (SaaS na Cloud)

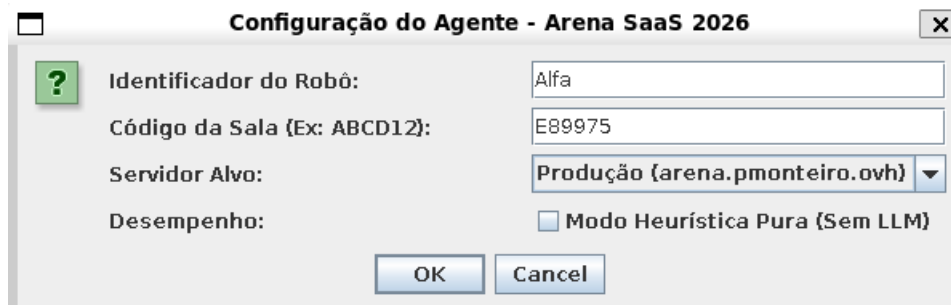
Hospedado num servidor na infraestrutura da OVH (<https://arena.pmonteiro.ovh>), este é o árbitro imparcial e o “Mundo Físico” do vosso agente. Desenvolvido em Python (FastAPI + NiceGUI), o servidor tem as seguintes responsabilidades:

- **Gestão de Estado e Física:** Mantém a grelha do mapa, as posições dos robôs, as colisões com as paredes e o nível de energia de cada participante em tempo real.
- **Motor de Regras:** Aplica as penalizações de dano em caso de ataques, gere o temporizador de 10 minutos da partida e executa o rigoroso sistema Anti-Flood (bloqueando agentes que façam chamadas sem respeitar os tempos de pausa).
- **Renderização 3D:** Oferece uma interface web (através do vosso browser) para que o Professor e os alunos possam assistir ao vivo, através de WebSockets, a locomoção dos agentes e os efeitos visuais da simulação.
- **Avaliação RAG:** Detém a base de dados dos códigos secretos dos cofres e valida se a palavra extraída pelo vosso modelo LLM está correta, atribuindo a vitória ou o dano consoante o resultado.

2.2. O Cliente Autónomo (Java - Execução Local)

Este é o programa que vocês vão desenvolver e executar localmente no vosso computador. Atuando como o “sistema nervoso central” do robô, o cliente Java faz a ponte entre o Servidor e a Inteligência Artificial. As suas funções são:

- **Telemetria:** Fazer pedidos HTTP GET regulares ao servidor da Arena para “ver” o que está em redor do robô (o radar).
- **Heurística e Memória:** Processar os dados recebidos para construir matrizes lógicas, como o Mapa de Calor (evitar andar em círculos) e a Lista Negra (ignorar cofres já falhados).
- **Orquestração:** Quando confrontado com um Terminal de Plasma, o Java descarrega o ficheiro do manual técnico do servidor, parte-o em pedaços (Chunks) e envia-os para o motor local de IA.
- **Ação:** Enviar o pedido HTTP POST final para a Arena com o movimento decidido (MOVER_NORTE, MOVER_SUL, etc.) ou a tentativa de código para abrir o cofre.



The image shows a Windows-style dialog box titled "Configuração do Agente - Arena SaaS 2026". It has a standard title bar with a minimize button, a maximize button, and a close button (X). The dialog contains four labeled text input fields: "Identificador do Robô:" with the value "Alfa", "Código da Sala {Ex: ABCD12}:" with the value "E89975", "Servidor Alvo:" with a dropdown menu showing "Produção {arena.pmonteiro.ovh}", and "Desempenho:" with a checkbox labeled "Modo Heurística Pura {Sem LLM}" which is currently unchecked. At the bottom of the dialog are two buttons: "OK" and "Cancel".

Fig 1. Exemplo de ecrã de início de um agente

2.3. O Motor de Inteligência Artificial (Ollama - Execução Local)

Em vez de sobrecarregar o servidor do Professor com cálculos de IA pesadíssimos e caros, o “cérebro” do agente vai correr na máquina de cada aluno, suportado pelo motor local Ollama e respondendo no porto localhost:11434. Este serviço é responsável por:

- **Vetorização (Embeddings):** Utilizando o modelo `nomic-embed-text`, transforma as frases do manual técnico e os enigmas dos cofres em arrays matemáticos multidimensionais para permitir buscas de semelhança.
- **Geração (LLM):** Utilizando o modelo otimizado para lógica `qwen2.5-coder:0.5b-instruct`, recebe o pedaço de texto filtrado (RAG) e o enigma, lendo a vossa instrução rígida de Prompt Engineering para gerar exclusivamente a chave alfanumérica necessária.

Resumo do Fluxo de Dados:

- O vosso Java (Local) pergunta ao Servidor (OVH): “O que estou a ver?”
- O Servidor (OVH) responde com um JSON: “Estás em cima de um cofre. O enigma é X.”
- O vosso Java (Local) interroga o Ollama (Local): “Transforma o enigma X em vetores e extrai a chave deste parágrafo.”
- O Ollama (Local) devolve ao Java (Local) a chave encontrada: “A chave é POLY-SEAL.”
- O vosso Java (Local) envia ao Servidor (OVH): “Quero desbloquear o cofre com a chave POLY-SEAL.”

2.4. Documentação da API (Swagger UI)

Num ambiente real de engenharia, raramente vos será dado um guião passo-a-passo sobre como comunicar com um servidor de terceiros. Em vez disso, ser-vos-á entregue a documentação da API.

O servidor da Arena 3D RAG dispõe de uma interface **Swagger UI** — uma documentação “viva” e interativa que descreve todos os *endpoints* (URLs de comunicação), métodos (GET/POST) e as estruturas dos ficheiros JSON esperados.

Como Aceder e Consultar: Podem explorar toda a “matemática” do servidor abrindo o vosso *browser* no seguinte endereço: <https://arena.pmonteiro.ovh/docs>

O que devem procurar no Swagger: Antes de escreverem a primeira linha do vosso cliente Java (`ArenaClient.java`), devem usar esta página para estudar os serviços críticos do jogo:

- **/register (POST):** Qual é o JSON que tenho de enviar para inscrever o meu robô numa sala?
- **/perceive (GET):** Qual é a estrutura do JSON gigante que o radar me devolve? Como se chamam as chaves (*keys*) que contêm as coordenadas dos cofres?
- **/action (POST):** Qual o formato exato da *string* de movimento para andar para o Norte?
- **/unlock (POST):** Como devo submeter a chave gerada pelo meu motor RAG para abrir um terminal?

Dica de Ouro: O Swagger não serve apenas para leitura. Podem clicar no botão **"Try it out"** em qualquer um dos serviços e simular pedidos HTTP diretamente no *browser*, inspecionando o resultado em tempo real. Isto poupar-vos-á dezenas de horas de depuração de código cego!

3. Preparação do Ambiente de Desenvolvimento (Setup)

Para que o vosso agente consiga raciocinar e comunicar com o servidor da Arena, precisam de preparar a vossa máquina local com as ferramentas de desenvolvimento e os motores de Inteligência Artificial adequados. Siga rigorosamente os passos abaixo.

3.1. O Ambiente Java e Dependências (Maven)

O vosso agente será programado em Java. Recomendamos a utilização da versão Java 11 ou superior (devido ao uso nativo do `java.net.http.HttpClient` para as chamadas REST).

O projeto deve ser gerido via Maven. Para que o vosso código consiga ler e escrever ficheiros JSON de forma eficiente nas comunicações com o servidor da Arena e com o Ollama, devem adicionar a biblioteca Gson da Google ao vosso ficheiro `pom.xml`:

```
<dependencies>
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
  </dependency>
</dependencies>
```

Nota: Certifiquem-se de que a vossa IDE (IntelliJ) atualiza o Maven após a inserção desta dependência.

3.2. Instalação do Motor de IA Local (Ollama)

O cérebro do vosso robô vai correr localmente. Para isso, vamos utilizar o Ollama, uma plataforma open-source otimizada para executar Modelos de Linguagem de grande e pequena escala de forma eficiente na vossa máquina.

Passo 1: Descarreguem e instalem o Ollama a partir do site oficial:
<https://ollama.com/download> (Disponível para Windows, macOS e Linux).

Passo 2: Obter os Modelos Necessários Após a instalação, abram o terminal do vosso computador (Linha de Comandos / PowerShell / Terminal) e executem os seguintes comandos para descarregar os dois modelos que o vosso código Java irá invocar:

O Modelo de Raciocínio / Extração RAG: Este é o SLM (Small Language Model) otimizado para tarefas de código e lógica, que vai ler os manuais e extrair as chaves.

```
ollama pull qwen2.5-coder:0.5b-instruct-q4_K_M
```

O Modelo de Vetorização (Embeddings): Este modelo serve apenas para matemática. Vai converter os parágrafos de texto do manual em coordenadas vetoriais para calcularmos distâncias semânticas.

```
ollama pull nomic-embed-text
```

3.3. Verificação de Prontidão do Sistema

Antes de começarem a programar as chamadas HTTP no vosso cliente Java, garantam que o Ollama está a correr em plano de fundo no vosso computador.

Por defeito, o Ollama inicia um servidor na porta 11434. Podem testar se está tudo operacional abrindo o browser e acedendo a: `http://localhost:11434`

Se a página exibir a mensagem “Ollama is running”, a vossa máquina está oficialmente pronta para iniciar o desenvolvimento do Agente Explorador!

3.4. Paradigma POO e Estrutura de Ficheiros Obrigatória

Este projeto não é apenas um guião de algoritmos; é um exercício de Arquitetura de Software. A vossa base de código deve seguir estritamente o paradigma de Programação Orientada a Objetos (POO).

É terminantemente proibido concentrar toda a lógica no método main ou criar classes monolíticas. Devem aplicar os princípios de Responsabilidade Única (SRP) e encapsulamento, separando os clientes HTTP, as interfaces gráficas e as estruturas de dados.

Para garantir esta organização, o vosso projeto tem de conter, no **mínimo**, a seguinte estrutura de ficheiros (classes) obrigatória:

- **AgenteExplorador.java:** A classe principal (Main). Atua como o orquestrador do ciclo de vida, contendo o loop infinito e delegando as tarefas (Perceção, Decisão e Ação) às restantes classes.
- **ArenaClient.java:** Uma classe dedicada exclusivamente a gerir as comunicações HTTP (GET/POST) com o servidor da Arena na OVH (registo, telemetria, ações e extração do manual).

- **OllamaClient.java:** Uma classe cliente independente responsável por gerir a comunicação com o motor de Inteligência Artificial local (geração de embeddings vetoriais e extração RAG via Prompts).
- **DocumentoVetorial.java:** O modelo de dados (Classe POO simples com Getters/Setters) que representa na memória RAM cada pedaço de texto do manual e o seu respetivo vetor matemático (Array de doubles).
- **PainelMapaCalor.java:** A classe responsável por encapsular todo o código visual e de desenho (Java Swing / Graphics2D) que mostra o dashboard de telemetria e o radar na vossa máquina local.

Nota: São totalmente encorajados a criar outras classes de suporte (por exemplo, MotorHeuristico.java ou modelos de dados adicionais para mapear o JSON), mas as cinco classes acima são o pilar estrutural exigido para a avaliação.

4. A Física da Arena: Regras e Mecânicas

Uma IA brilhante num chassis sem bateria não passa de um pisa-papéis tecnológico. Para que o vosso agente sobreviva e cumpra a missão, o vosso código deve compreender e respeitar as leis da física e da termodinâmica que regem o servidor da Arena.

4.1. Espaço Matricial e Locomoção

O mapa da Arena é gerado no servidor como uma grelha bidimensional invisível baseada em coordenadas (X, Y). Embora a visualização no vosso browser seja num ambiente 3D rico, o raciocínio matemático do vosso robô opera em 2D.

- **Movimentos Discretos:** O robô desloca-se bloco a bloco. No vosso código Java, os comandos de movimento autorizados pela API são estritamente: “MOVER_NORTE”, “MOVER_SUL”, “MOVER_ESTE” e “MOVER_OESTE”.
- **Obstáculos Sólidos:** A Arena possui Muros (`fixed_objects`). Se o vosso agente tentar mover-se para uma coordenada ocupada por um muro, o servidor bloqueará a ação e devolverá um estado de “bloqueado” por colisão. É da responsabilidade do vosso algoritmo mapear estes obstáculos e contorná-los no turno seguinte.

4.2. O Radar de Telemetria e Linha de Visão

O robô não tem acesso ao mapa completo da arena. Ele depende de um varrimento de radar obtido pelo endpoint de Perception (`/perceive`).

- **Raio de Visão:** O radar do agente tem um alcance máximo definido no servidor de 4.5 unidades de distância. Tudo o que estiver fora deste raio será invisível na matriz de dados recebida.
- **Oclusão Visível:** As paredes da Arena não bloqueiam apenas o movimento; elas bloqueiam o próprio sinal do radar. Se um Terminal de Plasma ou um recurso estiver escondido atrás de uma parede, o robô não o conseguirá “ver” até ter uma linha de visão desobstruída (ausência de intersecção com `fixed_objects`).

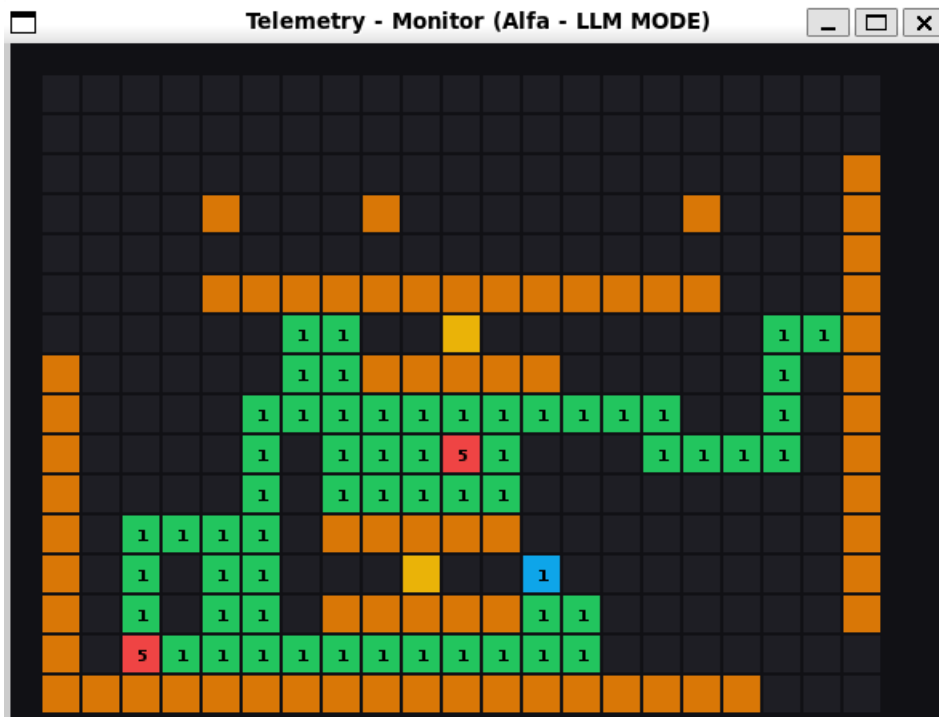


Fig 2. Exemplo da forma como o agente pode mapear o seu mundo, à medida que explora o mundo

4.3. Bateria Central (Gestão de HP)

A energia é o recurso mais crítico do jogo. O vosso robô entra na Arena com 200 HP (Pontos de Vida / Energia). Se este valor chegar a zero, o agente sofre morte térmica, sendo gerada uma lápide (“RIP”) no local e a vossa ligação à API é sumariamente terminada.

A matemática do consumo e regeneração de energia obedece às seguintes regras:

- **Custo de Movimento:** Cada passo válido dado pelo agente consome 1 HP da bateria.
- **Artefactos de Recarga:** Espalhadas pelo mapa estão pequenas esferas de energia (resources). Quando o robô partilha a mesma coordenada (X,Y) de um recurso, este é automaticamente coletado, regenerando 20 HP.
- **O Limite do Chassis:** A bateria do robô não consegue armazenar energia infinita. A capacidade máxima do tanque está limitada no servidor a 250 HP.
- **Recompensa de Missão:** Aceder a um Terminal de Plasma (vaults) e submeter o código RAG correto recompensa o agente com 100 HP imediatos (limitados aos 250 HP).
- **Penalização RAG:** Tentar “adivinhar” o código do cofre através de força bruta ou submeter um output alucinado da IA resulta num choque elétrico devolvido pelo terminal. Cada tentativa falhada drena 10 HP ao agente.

4.4. Ações Destrutivas e Morte

As consequências na arena são irreversíveis durante uma sessão de jogo. Se o agente for destruído (seja por falta de energia, por falhar demasiados cofres, ou em contexto de batalha), não existe respawn automático. O agente é imediatamente expulso da matriz da sala e o vosso cliente Java receberá o estado “eliminado”. O professor terá de instanciar ou reiniciar o cenário a partir da sua Torre de Vigia (Admin Panel) para vos permitir uma nova tentativa.

5. O Ciclo de Vida do Agente (Percepção, Decisão e Ação)

Para que o vosso robô atue de forma autónoma sem qualquer intervenção humana, a classe `AgenteExplorador.java` deve ser desenhada com base num ciclo infinito (`while (true)`). Dentro deste ciclo, o vosso algoritmo irá executar continuamente o paradigma clássico da Inteligência Artificial: **Sense-Think-Act (Percepção, Decisão, Ação)**.

5.1. A Fase de Percepção (Sense)

No início de cada iteração do ciclo, o robô está “cego”. A primeira tarefa é interrogar o servidor através do vosso cliente HTTP para saber o que o rodeia.

O servidor devolverá um grande objeto JSON contendo a matriz de telemetria atual. O vosso código (usando a biblioteca Gson) terá de extrair informações vitais de arrays específicos, nomeadamente:

- `game_started` e `game_over`: Indicadores booleanos. Se o jogo ainda não começou, o agente deve entrar num estado de espera (Lobby). Se o jogo terminou, deve desligar os motores e encerrar o programa.
- `o_meu_estado`: Informa as vossas coordenadas atuais (X,Y) e a energia (HP) restante.
- `objetos_fixos`: Lista das paredes que estão no vosso raio de visão.
- `recursos_no_mundo`: Coordenadas das esferas de energia visíveis.
- `cofres_no_mundo`: Coordenadas dos Terminais de Plasma e, mais importante, o texto do enigma (`terminal_desafio`) caso estejam literalmente em cima dele.

5.2. A Fase de Decisão (Think) e Memória Heurística

Ter a lista de paredes visíveis não basta; o robô precisa de saber analisá-las para não bater de frente contra elas. É nesta fase que constroem a inteligência de navegação.

Começam por definir as quatro intenções base: `MOVER_NORTE`, `MOVER_SUL`, `MOVER_ESTES`, `MOVER_OESTE`. A partir daqui, o vosso algoritmo deve filtrar esta lista:

- **Verificação de Colisões:** Iterar sobre os `objetos_fixos` do JSON. Se houver uma parede na coordenada (X+1, Y), o vosso código deve remover sumariamente `MOVER_ESTES` da lista de ações permitidas.
- **O Mapa de Calor (Heatmap):** Se o robô não tiver memória, corre o risco de ficar a andar para a frente e para trás no mesmo corredor infinitamente. Para resolver isto,

devem criar um dicionário (`Map<String, Integer>`) em Java que guarde o histórico de visitas (`historicoVisitas`). Cada vez que o robô pisa uma coordenada, incrementa o contador. Na hora de decidir o próximo passo, o algoritmo deve olhar para as opções adjacentes e escolher o caminho “mais frio” (com menor número de visitas).

- **Atração Magnética:** Se o JSON relatar que existe um cofre ou energia por perto, o vosso algoritmo de navegação heurística deve sobrepor-se ao mapa de calor e traçar a rota mais direta para o alvo.

5.3. A Fase de Ação (Act) e Fila de Comandos

Com a decisão tomada, o vosso código emite a chamada REST (POST) para o servidor da Arena (`/action`), submetendo o movimento ou ação escolhida.

O servidor processará o pedido de acordo com as leis da física e devolverá um JSON de resposta. Este JSON dirá se o resultado foi um “sucesso” (atualizando a vossa posição) ou se foram “bloqueado” / “eliminado”. O vosso código deve estar preparado para ler este feedback e reagir em conformidade no turno seguinte.

Dica de Arquitetura - Reflexos Táticos: Para otimizar o tempo de reação, o vosso agente pode usar uma Fila de Ações Planeadas (`filaAcoesPlaneadas`). Por exemplo, se após abrir um cofre o robô quiser abandonar a área rapidamente, pode injetar 2 ou 3 movimentos predeterminados nesta fila. No início do ciclo `while`, o agente verifica se há comandos pendentes na fila; se houver, executa-os imediatamente (despachando as ações reflexas sem ter de passar por todo o pesado processamento heurístico do JSON).

6. O Desafio RAG: Terminais de Plasma e LLM

A locomoção e a sobrevivência são apenas o prelúdio. O verdadeiro objetivo do vosso agente é desbloquear os Terminais de Plasma espalhados pelo mapa. Para o fazer, terão de implementar um pipeline completo de RAG (Retrieval-Augmented Generation) de raiz no vosso código Java.

6.1. Detecção e Ativação do Terminal

No vosso varrimento de telemetria (`/perceive`), recebem o array `cofres_no_mundo`. Quando as vossas coordenadas (X,Y) coincidirem exatamente com as coordenadas de um cofre, o vosso robô estará posicionado sobre o terminal. Nesse exato momento, o JSON recebido passará a incluir o campo `terminal_desafio`. Este campo contém um enigma ou relatório de avaria em linguagem natural (ex: “Avaria grave térmica no gerador”). É este o gatilho que inicia o processo cognitivo.

6.2. Gestão de Dados e Embeddings (Fase de Retrieval)

Para que o agente saiba resolver o enigma, ele precisa de estudar. Logo após o registo inicial na Arena, o vosso código Java deve descarregar o manual técnico através do endpoint fornecido pelo servidor.

- **Chunking:** O vosso programa deve ler esse documento e dividi-lo em blocos lógicos (ex: separar por quebras de linha `\n`).
- **Vetorização:** De seguida, devem enviar cada bloco de texto para o motor local do Ollama, chamando o modelo de Embeddings (`nomic-embed-text`). O modelo não vos devolve texto, mas sim um longo array de números decimais (vetores matemáticos) que representam o “significado” daquela frase.

O vosso agente deve guardar estes objetos em memória (na RAM) enquanto a partida decorrer.

6.3. Pesquisa Semântica (Cosine Similarity)

Quando o agente pisa o cofre e lê o `terminal_desafio`, ele não sabe a resposta de imediato. A magia matemática acontece aqui:

- O vosso Java envia o enigma para o Ollama e gera um novo vetor para a pergunta.
- Devem programar uma função de **Semelhança de Cossenos (Cosine Similarity)** no vosso código. Esta fórmula matemática compara o vetor da pergunta com todos os vetores dos parágrafos do manual que guardaram na memória.
- O parágrafo que devolver o valor mais próximo de 1.0 é o contexto semanticamente mais relevante. Acabaram de encontrar a agulha no palheiro!

6.4. Prompt Engineering e Extração (Fase de Generation)

Agora que têm a pergunta e a secção correta do manual, é hora de acionar o “cérebro” (Small Language Model). Têm de enviar um pedido ao Ollama usando o modelo `qwen2.5-coder:0.5b-instruct`.

No entanto, o modelo é tagarela por natureza e tentará responder com frases longas como “Com certeza, analisando o texto...”. A API da Arena não aceita frases, aceita apenas a chave alfanumérica isolada.

Para forçar a submissão perfeita, o vosso código Java terá de usar a técnica de **ChatML (Chat Markup Language)** com formatação rígida:

- `<|im_start|>system:` Onde injetam as restrições negativas (ex: “És um extrator de dados. Responde apenas com a palavra-chave. Não inventes.”).
- `<|im_start|>user:` Onde passam a “Secção do Manual” e o “Enigma”.
- `<|im_start|>assistant\n:` Onde fecham a tag, forçando o LLM a iniciar a sua resposta diretamente pela palavra que queremos, anulando alucinações introdutórias.
- **Determinismo:** Configurem o vosso pedido HTTP ao Ollama para que a temperatura seja `0.0`. Isto erradica a criatividade e transforma a IA numa máquina lógica implacável.

6.5. Submissão, Lista Negra e Prevenção de Loops

O último passo é submeter o código extraído pelo LLM ao endpoint de desbloqueio da Arena (/unlock).

- Se for sucesso (status: "sucesso"): O cofre abre-se, a bateria do agente é massivamente recarregada (+100 HP) e vocês avançam no jogo.
- Se for falha (status: "falha"): O robô leva um choque elétrico (-10 HP). Se a semântica falhar e a resposta estiver errada, o vosso agente não pode ficar a tentar infinitamente. Devem programar uma “Lista Negra” (Blacklist) no vosso Java (cofresFalhados). Quando um cofre falha, o agente memoriza essas coordenadas como inacessíveis e força um movimento de fuga para o bloco adjacente, de modo a prosseguir a exploração sem esgotar as suas baterias.

7. Regras de Ouro do Servidor e Sobrevivência

O servidor central que gere a Arena não é apenas um anfitrião; ele possui mecanismos de autodefesa implacáveis para garantir a estabilidade da infraestrutura para todos os alunos em simultâneo.

Se o vosso código for egoísta ou mal comportado, o servidor irá penalizá-lo e expulsá-lo da rede.

7.1. O Temporizador da Operação (Timeout)

As missões na arena não duram para sempre. Para evitar que bots fiquem presos infinitamente nos labirintos, cada sessão instanciada pelo Professor tem um limite de tempo global fixado em 10 minutos (600 segundos).

O relógio da arena inicia a sua contagem decrescente assim que o Professor dispara o início do jogo na interface central.

Se o tempo chegar ao fim (00:00), a partida é abruptamente encerrada. No modo “Missão”, o resultado será “Missão Falhada”. No modo “Batalha”, se existirem vários sobreviventes, o sistema ditará a vitória de forma algorítmica àquele que tiver conservado a maior percentagem de energia (HP). Se houver empate, será exigida uma revanche.

7.2. O Sistema Anti-Flood e Defesa DDOS

O erro mais amador que podem cometer ao desenhar a Inteligência Artificial do vosso agente é criar um ciclo `while(true)` sem pausas de processamento. Um robô a disparar pedidos HTTP ininterruptamente a 0 milissegundos atua como um ataque de negação de serviço (DDoS), estrangulando o processador do servidor Ubuntu.

Para impedir isto, a Arena tem um “Tarpit” (fosso de breu) defensivo:

- **Limite de Velocidade:** O servidor exige que exista um intervalo estrito entre as vossas chamadas de perceção/ação.

- **O Castigo (Blacklist Temporária):** Se o vosso Java enviar pedidos mais rápido do que a tolerância mínima permitida, o servidor irá travar o vosso robô e devolver um JSON com o estado “bloqueado”. O castigo é severo: o vosso agente ficará totalmente paralisado durante 5 segundos, incapaz de fugir ou atacar, deixando-o à mercê dos inimigos.
- **A Solução Mandatória:** É absolutamente obrigatório que o vosso ciclo lógico em Java possua a instrução `Thread.sleep()` no final de cada iteração para dar “folga” à rede.

7.3. Tolerância de Rede e Jitter (Flutuações Wi-Fi)

Na teoria, se o servidor exige uma pausa mínima de 200ms entre pedidos, programar o vosso `Thread.sleep(200)` resolveria o problema. Na prática do mundo real, a rede Wi-Fi da universidade tem variações de latência (Jitter).

Se enviarem o “Pedido A” e logo a seguir o “Pedido B” com exatamente 200ms de diferença, o “Pedido A” pode ficar retido num router local por um segundo, e subitamente os dois pacotes chegam ao servidor ao mesmo tempo, acionando o bloqueio Anti-Flood injustamente.

- **Zona Tampão (Buffer):** Os engenheiros de software profissionais resolvem isto com margens de segurança. O vosso `Thread.sleep()` de rotina não deve ser inferior a 350ms ou 400ms. Isto permite que a rede acomode engarrafamentos de pacotes sem violar as regras de produção do servidor.
- **Quedas de Ligação:** Se observarem um erro no terminal Java a dizer `request timed out`, isso não é o servidor a bloquear-vos; é a vossa ligação à Internet que sofreu um micro-corte e abortou a chamada HTTP. O vosso código deve estar preparado com blocos `try-catch` robustos para absorver estas anomalias e voltar a tentar no turno seguinte sem que a aplicação feche (Crash).

8. Modos de Operação

O vosso agente será submetido a avaliações distintas na Arena. O servidor da OVH está configurado com diferentes tipos de cenário (`tipo_jogo`). Para obter a nota máxima, o vosso código Java (`AgenteExplorador.java`) tem de ser suficientemente versátil para perceber o contexto em que foi inserido e alterar a sua “Diretriz Tática” dinamicamente.

Existem dois modos principais de operação:

8.1. Modo de Exploração (Missão)

Este é um cenário focado puramente em lógica, navegação e processamento de linguagem natural (RAG).

- **Objetivo:** Encontrar e desbloquear todos os Terminais de Plasma (cofres) presentes no mapa antes que os 10 minutos terminem.

- **Regras de Engajamento:** O combate físico está estritamente desativado pelo servidor. Se o vosso robô tentar investir contra outro agente neste modo, o servidor devolverá a mensagem “Combate desativado!” e a ação será bloqueada.
- **O Foco do Vosso Algoritmo:** Neste modo, devem desligar as vossas rotinas agressivas. O agente deve focar-se em otimizar a exploração usando o Mapa de Calor (historicoVisitas), fazer a digestão do manual técnico em tempo real e resolver os enigmas cirurgicamente para não desperdiçar bateria com penalizações de tentativas erradas.

8.2. Modo Competitivo (Battle Royale)

Aqui a arena transforma-se num ambiente hostil de sobrevivência extrema, onde múltiplos grupos de alunos largam os seus agentes em simultâneo.

- **Objetivo:** Ser o último robô sobrevivente. Se o tempo de 5 minutos expirar e houver mais do que um sobrevivente, ganha aquele que tiver mais energia (HP) no momento do apito final.
- **Mecânica de Combate:** O combate está ativo. Se o vosso robô se mover para a mesma coordenada (X,Y) onde já se encontra um robô rival, é ativado um ataque de investida (Burst/Melee).
 - O ataque custa-vos 1 HP de energia cinética.
 - O impacto causa 15 HP de dano direto ao robô inimigo.
 - O servidor emite um efeito visual de explosão no mapa e regista a ocorrência no registo de auditoria global.
- **O Foco do Vosso Algoritmo (Luta ou Fuga):** Analisar o JSON na chave outros_robots é vital. Se o vosso radar detetar um rival a menos de 2 blocos de distância, o vosso código tem de tomar uma decisão crítica baseada no Delta de Energia (A vossa energia atual vs. a energia do rival detetado no radar):
 - **Fuga (Flight):** Se o vosso robô tem 40 HP e o inimigo tem 120 HP, investir é suicídio. O vosso algoritmo deve injetar ações de fuga na vossa Fila Planeada (mover na direção oposta ao inimigo) para salvar a vida.
 - **Ataque (Fight):** Se tiverem vantagem energética, o vosso algoritmo deve trancar as coordenadas do alvo e emitir comandos na direção dele repetidamente até o desintegrar.
- **Vantagem Tática RAG:** Mesmo na batalha, os cofres continuam no mapa. Conseguir fugir de uma luta, encontrar um cofre e extrair a chave correta usando o modelo qwen2.5-coder dá-vos +100 HP de cura instantânea, o que pode virar o rumo da batalha a vosso favor!

8.3. Salas de Treino Pré-Exame (Sandbox Oficial)

Para garantir que o vosso agente está perfeitamente afinado antes da avaliação final, o servidor central da Arena disponibiliza cenários de treino contínuos e abertos 24 horas por dia.

Para poderem testar o vosso código livremente antes do exame, sem medo de penalizações definitivas, o Professor configurou duas instâncias de teste persistentes. Podem criar instâncias de novos jogos de teste, usando os códigos abaixo.



Fig 3. Consola Web do Servidor, onde os alunos podem criar jogos ou aceder aos mesmos para ver como os seus agentes estão a interagir com os mundos.

Devem utilizar um dos seguintes códigos de acesso para os vossos testes de desenvolvimento:

-  **Código da Sala: aluno_treino_2026 (Cenário Missão - Treino)**
 - **Objetivo:** Laboratório pacífico. Usem esta sala para testar e depurar a vossa navegação heurística, o mapeamento de obstáculos e, acima de tudo, para calibrar a precisão matemática do vosso *pipeline* RAG com o modelo LLM ao abrirem os cofres. Não há risco de serem atacados.
-  **Código da Sala: arena_batalha_global (Cenário Batalha - Treino)**
 - **Objetivo:** O ringue de testes competitivo. Entrem nesta sala quando o vosso robô já souber andar sozinho. Aqui o combate está ativo. É o local ideal para testarem o comportamento do vosso algoritmo quando o radar deteta inimigos (lógica de *Fight or Flight* / Combate ou Fuga) e avaliarem se o vosso `Thread.sleep()` aguenta o caos de múltiplos alunos ligados ao mesmo tempo. Neste cenário podem lançar mais que um agente vosso, ou simular contra agentes dos vossos colegas.

9. Fases de Entrega e Critérios de Avaliação

O sucesso neste projeto não se mede apenas pela capacidade de o robô sobreviver na Arena, mas também pelas boas práticas de Engenharia de Software aplicadas durante a sua construção e pela transparência no uso de ferramentas modernas de desenvolvimento.

9.1. Submissão de Código e Controlo de Versões (GitHub)

A entrega central do vosso projeto será realizada através de um repositório no GitHub. O código-fonte do vosso agente (AgenteExplorador.java e restantes classes de suporte) deve ser devidamente versionado.

- **Acesso do Professor:** O repositório deve ser privado, mas têm obrigatoriamente de convidar o professor como colaborador/leitor, dando acesso através do email pscm@ipt.pt. Sem este acesso, o projeto não será avaliado.
- **Commits Progressivos:** A engenharia de software é iterativa. Será escrutinado o histórico do Git. Exige-se que os commits sejam realizados de forma progressiva ao longo das semanas de desenvolvimento (projetos com um único "commit gigante" na véspera da entrega sofrerão pesadas penalizações).
- **Trabalho de Equipa (Duplas):** Tipicamente, os grupos são constituídos por 2 elementos. O desenvolvimento tem de ser equitativo. O histórico de commits deve refletir claramente as contribuições e a participação de ambos os elementos do grupo. O GitHub será a ferramenta primária para aferir quem programou o quê.

9.2. Transparência na Geração de Código e Uso de LLMs

Sendo este um projeto focado em Inteligência Artificial, o uso de LLMs (como o ChatGPT, Claude, Gemini ou GitHub Copilot) para auxílio no desenvolvimento, explicação de erros ou otimização de funções é permitido e encorajado. No entanto, a transparência tem de ser absoluta e **individual**.

Como o trabalho é feito em equipa, o Professor exige aferir o nível de pesquisa e *Prompt Engineering* de **cada aluno separadamente**. Por isso, dentro do vosso repositório GitHub, devem submeter os ficheiros de **Auditoria de IA** seguindo estas regras estritas:

- **Ficheiros Individuais:** Não é aceite um único ficheiro fundido para o grupo. **Cada aluno do grupo tem de entregar o seu próprio ficheiro** com as conversas que teve com a IA.
- **Formato e Nomenclatura Exigida:** O documento deve ser entregue estritamente em formato **PDF** ou **Markdown (.md)**. O nome do ficheiro tem de identificar claramente o grupo e o aluno, por exemplo:

- Auditoria_IA_Grupo4_12345_JoaoSilva.pdf
- Auditoria_IA_Grupo4_67890_MariaSantos.pdf

- **Cabeçalho de Identificação Obrigatório:** Logo no topo da primeira página do vosso ficheiro individual, tem de constar um cabeçalho explícito:

- **Número do Grupo:** (Exemplo: *Grupo 4*)
- **Identificação do Aluno:** Nome completo e Número de Aluno (Exemplo: *João Silva - nº 12345*).

- **Apresentação do Conteúdo:** O histórico da conversa não pode ser um mero aglomerado de texto confuso. O documento deve estar estruturado de forma a distinguir claramente:

- **O vosso Prompt:** A pergunta, o contexto ou o erro que forneceram à IA.
- **A Resposta da IA:** O código ou explicação gerada pelo modelo.

- O objetivo desta entrega individual é avaliar a capacidade de raciocínio de cada elemento: o Professor irá aferir se o aluno sabe fazer as perguntas certas e se compreende genuinamente o código que a IA sugeriu antes de o integrar no projeto comum.

9.3. Avaliação ao Vivo (A Grande Final)

A nota final não será dada apenas por ler o vosso código, mas por vê-lo em ação. Na data agendada para a avaliação, o professor levantará instâncias reais da Arena no servidor de produção da OVH (arena.pmonteiro.ovh).

- Os grupos ligarão os seus agentes ao servidor perante a turma.
- Serão testados num modo “Missão Solo” para avaliar a pureza do vosso motor RAG e a capacidade de encontrar e decifrar cofres sem interferência.
- Serão atirados para um modo “Battle Royale” contra os agentes de outros grupos para avaliar a vossa heurística de combate, gestão de energia, fuga e tolerância a falhas de rede.

9.4. Distribuição dos Critérios de Avaliação

A vossa nota será ponderada com base nas seguintes dimensões de engenharia e lógica:

1. Arquitetura de Software e POO (O Anti-Padrão "Classe Deus"): A avaliação incidirá fortemente no cumprimento rigoroso do paradigma de Programação Orientada a Objetos. A presença da estrutura mínima obrigatória de ficheiros é exigida. **Será aplicada uma penalização severa a projetos que apresentem o anti-padrão da "Classe Deus" (God Class).**

O que é a Classe Deus? É um erro clássico e crónico de arquitetura onde uma única classe (geralmente a main) concentra em si todas as responsabilidades do sistema, tornando-se um monstro de milhares de linhas, impossível de manter ou testar. No contexto deste projeto, uma "Classe Deus" seria um ficheiro `AgenteExplorador.java` que, sozinho, tenta controlar o ciclo de vida do jogo, constrói os *headers* HTTP para a Arena, formata o JSON do Ollama, calcula a matemática vetorial e ainda desenha a interface gráfica. *A Regra:*

Respeitem o Princípio da Responsabilidade Única (SRP). Quem comunica com a rede não desenha ecrãs; quem processa texto de IA não lida com a locomoção na matriz.

2. Pipeline RAG e Prompt Engineering: O agente consegue extrair chaves precisas usando o Ollama local? Lida bem com falhas (Lista Negra de cofres)? A formatação ChatML no Java está bem construída e determinística?

3. Heurística e Decisão Espacial: O Mapa de Calor previne que o robô ande em círculos? O agente respeita os limites do radar? Evita bater nas paredes e sabe traçar rotas de fuga contra inimigos superiores?

4. Resiliência e Tolerância a Falhas: O código possui `Thread.sleep()` bem calibrado? Sobrevive a castigos do servidor Anti-Flood? O programa usa blocos try-catch robustos para não "crashar" se a ligação à rede for momentaneamente interrompida?

5. Git e Colaboração: A evolução dos *commits* faz sentido, é progressiva e reflete o esforço global e equitativo de todos os elementos do grupo?

6. Transparência IA: A documentação das interações com os LLMs na geração do código está completa e demonstra um uso crítico e inteligente destas ferramentas?